BudgetLab 技术架构与模块功能说明

预算感知 证据约束的表格分类 Research Agent

基于仓库当前实现 | 项目版本 0.1.0 | 2026 年 11 月 14 日

文档目的

本文面向需要理解、维护、演示或扩展 BudgetLab 的开发人员和项目成员，说明系统的整体技术路线、端到端功能、核心模块职责、数据与证据边界、运行产物以及当前限制。重点解释各模块为什么存在、如何协作，以及哪些决策由程序控制、哪些内容允许由 LLM 提出。

核心结论

- BudgetLab 是受控机器学习实验系统。LLM 只负责每轮提出两个结构化、可证伪的实验假设，不训练、不打分、不决定结果。
- 数据哈希、可用字段、切分种子、评价指标、最小有效提升和预算在第一次提案前冻结，后续实验必须服从同一契约。
- 研究阶段只读取 development 数据；密封 holdout 只能在交付冻结后由 evaluation 模块读取一次。
- 训练和 LLM 调用均先预留预算、执行后结算；无法确认的消耗进入 unknown，并通过事件账本和产物文件形成证据链。

阅读导航

章节	内容
1	系统定位 功能范围与技术栈
2	总体架构 信任边界与端到端流程
3	核心代码模块逐项说明
4	关键业务流程和状态变化
5	配置 产物 测试和运行方式
6	限制与扩展建议

1 系统定位与功能范围

1.1 要解决的问题

BudgetLab 面向表格二分类任务，将提出研究方向和验证研究结论拆分为两个相互制约的部分。提案器依据开发集画像、历史实验和剩余预算提出模型、预处理或特征动作变化；控制器负责合法性检查、去重、训练、指标计算、晋级、停止、确认和最终交付。自然语言推理可以参与搜索，但性能数字和结论必须来自可复现的程序计算。

1.2 当前已实现功能

能力	实现说明	主要模块
任务准备	加载原始数据，固定切分开发集和密封 holdout，记录文件哈希	tasks data
场景约束	场景卡定义预测时点和字段可用性，排除结果后字段	tasks contracts
实验提案	确定性 scripted 提案器和 Pydantic AI 结构化提案器	proposers
搜索对照	随机搜索与工具搜索共享同一合法空间	search controller
真实训练	sklearn Pipeline 在独立子进程执行交叉验证或全量重训	pipeline runner worker
程序裁决	同折配对差值、噪声地板和未见切分种子确认	evaluator controller
预算治理	预留和结算 Token 请求 成本 实验 拟合及时间	budget
审计交付	事件账本、上下文、日志、模型、报告和一次性 holdout	ledger reporting evaluation

1.3 当前未完成范围

- 尚未完成正式 benchmark 矩阵批量运行器、统一曲线汇总和结果对比页面。
- 尚未完成只读可视化 UI、演示 PPT 和经验库。
- 真实托管模型请求尚未形成已验证结果；当前接口路径通过本地 FunctionModel 测试。
- 主要任务是 Bank Marketing，乳腺癌数据仅用于工程冒烟；尚未接入 MLE-bench 或 Kaggle。
- 训练子进程提供超时和线程限制，但不是操作系统安全沙箱。

1.4 技术栈

类别	技术	用途
运行时	Python 3.11 及以上	控制器 数据处理 训练和命令行
数据	pandas 2.2.2 NumPy 1.26.4	表格处理 统计画像 数值计算
机器学习	scikit-learn 1.5.1 joblib 1.4.2	Pipeline 交叉验证 模型持久化
契约	Pydantic 2	冻结 Schema 判别联合类型 严格校验
LLM	pydantic-ai-slim 2.68.0	结构化输出 重试 usage 和 OpenAI 兼容接口
工程质量	pytest ruff GitHub Actions	测试 静态检查 跨平台 CI

2 总体技术架构

2.1 分层架构

层次	职责	组成
接口与配置	接收命令 读取 YAML 输出 JSON	cli configs
任务与数据	加载任务 解释场景卡 切分数据 生成画像	tasks data profile
契约	严格类型 冻结协议 校验规格 稳定哈希	schemas contracts
研究编排	创建 run 校准 探索 停止 确认 冻结	controller state
提案与搜索	脚本或 LLM 提案 Critic 随机和工具搜索	proposers search
训练与裁决	Pipeline 子进程 超时 指标和判定	pipeline runner worker evaluator
治理与审计	资源预算 原子写入 账本 代码环境指纹	budget ledger provenance
交付与评测	报告 模型冻结 holdout 复现和回放	reporting evaluation controller

2.2 端到端数据流

1. prepare 按固定种子切分 development 与 sealed holdout，并生成 manifest。
2. research 创建 run，保存配置、Git 与环境信息；加载 development 和场景卡得到合法字段。
3. 运行基线并在额外种子上估计波动，min_effect 取配置下限与噪声地板的较大值，然后冻结 contract。
4. 候选先经过 Schema、字段、比较器、证据引用和重复规格检查。
5. Runner 预留预算并启动 worker；worker 在 development 上执行交叉验证并返回每折分数。
6. Evaluator 计算同折差值和 verdict，ResearchState 更新最优实验与停滞状态。
7. 探索结束后，最优通过候选在三个未见种子上与基线再次比较。
8. confirmed_pass 才替代基线交付；模型全量重训并冻结后，evaluate 才能一次性读取 holdout。

2.3 关键控制边界

边界	控制者	防护方式
指标和裁决	程序	提案 Schema 无 verdict 字段，分数由 evaluator 计算
字段可用性	场景卡和契约	候选只能引用 approved_feature_columns
数据隔离	模块与文件路径	研究代码只调用 load_development，holdout 集中在 evaluation
资源使用	Budget	先 reserve 后 settle，无 usage 的失败进入 unknown
实验一致性	Contract 和 spec_hash	契约冻结，列顺序规范化后稳定哈希去重
最终交付	Controller 和 Evaluation	确认、全量重训、模型哈希和单次评测

3 核心模块说明

3.1 接口 配置 数据与契约

命令行入口 budgetlab cli.py

模块定位：提供统一命令行并把内部结果转换为稳定 JSON。

- doctor 检查配置和依赖；prepare 创建开发集与密封 holdout。

- research 执行完整研究；probe-llm 验证最终结构化输出 Schema。
- evaluate reproduce replay 分别负责最终评测、冻结规格重训和事件回放。

输入与输出：输入为命令参数和 YAML；输出为标准输出 JSON 以及 data 或 runs 产物。

技术要点：CLI 不实现裁决规则，并对 evaluation 与 LLM 组件采用延迟导入。

运行配置 configs

模块定位：以数据文件控制任务、种子、预算、循环策略和 LLM 参数。

- bank_marketing.yaml 用于当前开发和演示任务。
- night_smoke.yaml 用乳腺癌数据快速验证完整管线。
- LLM 请求和 Token 默认均为零，改为正数才授权真实调用。

输入与输出：Controller 将配置转换为 ProtocolSpec、BudgetLimits 和 LoopPolicy。

技术要点：配置与代码分离，便于冻结 benchmark 条件并比较不同系统。

任务注册与场景卡 budgetlab tasks

模块定位：统一适配数据集，并把预测时点和字段可用性作为可信业务输入。

- TaskDefinition 定义任务 ID、目标列、主指标、holdout 比例与加载函数。
- 场景卡为字段标记 available 或 post_outcome，approved_features 只保留预测时点可见字段。
- Bank Marketing 使用固定 URL 与 SHA256；乳腺癌任务读取 sklearn 内置数据。

输入与输出：输入为 raw 目录和 cards YAML；输出 features、target、任务元数据及合法字段。

技术要点：字段可用性不写死在训练代码中，新增任务必须附完整场景卡。

数据准备与切分 budgetlab data.py

模块定位：建立开发数据与最终评测数据的文件级隔离。

- prepare_task 分层切分并保存 development.csv、holdout.csv 和 manifest.json。
- load_development 只读取开发集，并返回行 ID、数据哈希和目标列。
- split_manifest 保存各随机种子和每折验证行 ID，支持同折复核。
- numeric_minima 为 log1p 等动作提供数值域校验。

输入与输出：输入为 TaskDefinition 和 data_root；输出 DatasetBundle、文件哈希和 split manifest。

技术要点：这是同机文件边界而不是操作系统沙箱，prepare 是唯一负责切出 holdout 的路径。

开发集画像 budgetlab profile.py

模块定位：生成供提案器理解数据的紧凑、确定性 EDA。

- 统计字段类型、缺失率、唯一值、范围、偏度和高频值。
- 识别可能的极端哨兵值，例如 pdays 的 999。
- 对可用字段计算分组正例率，不向提案器提供不可用字段的关联信息。

输入与输出：输入为 development 特征、标签、场景卡；输出 eda.json。

技术要点：所有统计均来自 development，属于提案证据而不是模型评价结果。

严格数据模型 budgetlab schemas.py

模块定位：集中定义配置、提案、实验、预算、评测和契约对象。

- StrictModel 禁止额外字段、对象冻结并拒绝 NaN 与 Infinity。
- ExperimentSpec 约束三类模型和四类特征动作。
- ProposalBatch 强制每轮恰好两个候选；Hypothesis 必须声明证据和比较器。
- ProtocolSpec、TaskContract、ExecutionResult、EvaluationRecord 构成核心契约。

输入与输出：被控制器、提案器、训练器和评测器共享，可稳定序列化为 JSON。

技术要点：LLM 无法输出 candidate_pass，因为该字段不属于提案 Schema。

契约冻结与规格校验 budgetlab contracts.py

模块定位：将可配置协议转换为不可变契约并阻止非法实验。

- canonical_json 和 stable_hash 生成稳定 SHA256。
- freeze_contract 校验 calibration 与 protocol 归属并冻结 min_effect。
- canonical_spec 对列列表排序，使仅顺序不同的规格视为重复。
- validate_experiment_spec 检查模型白名单、字段、动作顺序、数值域和文本兼容性。

输入与输出：输入 ProtocolSpec、CalibrationReport 和 ExperimentSpec；输出 TaskContract 或明确错误。

技术要点：非法规格直接拒绝，不做静默降级、参数修补或替换。

3.2 编排 提案 训练与治理

主控制器 budgetlab controller.py

模块定位：编排一个 run 的全部阶段，是程序决策中心。

- setup_run 保存来源信息，执行基线与校准并冻结契约。
- explore_random 和 explore_loop 实现三种研究系统。
- screen_candidates、choose_candidates、stop_reason 管理筛选、运行选择和停止。
- run_experiment 连接训练、基线比较、前一最优比较和假设比较。
- finalize 执行未见种子确认、全量重训、模型哈希和交付冻结。

输入与输出：输入配置、系统类型、Proposer 和 Critic；输出完整 run、summary、 report 和模型。

技术要点：Controller 保持唯一决策权，LLM 只能提供 ProposalBatch 或 CriticAdvice。

研究状态 budgetlab state.py

模块定位：维护实验历史、当前最优、重复映射和停滞状态。

- ExperimentRecord 汇总规格、执行结果、多个比较结论、耗时和累计 Token。
- ResearchState 维护 seen、best_id、stagnation、empty_rounds 与工具搜索状态。
- add 只有候选超过前一最优且达到 min_effect 时才返回有效改进。
- compact_spec 删除默认噪声，缩短上下文和报告。

输入与输出：输入程序计算的实验结果；输出当前研究状态。

技术要点：所有状态字段都来自程序计算，不接收 LLM 自报的分数或判定。

提案器与 Critic budgetlab proposers.py

模块定位：把研究上下文转换为两个结构化候选，并在停滞时给出只读建议。

- build_context 只暴露合法字段、开发集画像、历史实验、规则反馈和剩余预算。
- ScriptedProposer 确定性尝试类别权重、模型切换、哨兵处理和参数调整。
- PydanticAIProposer 与 Critic 使用严格结构化输出和可配置 output mode。
- CallOutcome 携带 output、实际 usage、 unknown、详情和错误。

输入与输出：输入压缩研究上下文；输出 ProposalBatch 或 CriticAdvice 及资源使用。

技术要点：LLM 看不到 sealed holdout，也不能执行训练；无 usage 的失败请求不会记为免费。

合法空间采样 budgetlab search.py

模块定位：为随机对照和规则工具搜索提供同一套零 Token 采样器。

- baseline_spec 定义标准化数值特征的逻辑回归基线。
- SpecSampler 在模型、类别权重、超参数和特征动作之间采样。
- sample_new 避免返回已运行规格。

- 只有画像支持时才采样 log1p、哨兵替换或文本分支。

输入与输出：输入冻结契约、EDA 和随机种子；输出 ExperimentSpec。

技术要点：对照组与 BudgetLab 工具搜索共享代码，降低搜索空间偏差。

机器学习 Pipeline budgetlab pipeline.py

模块定位：把实验规格转换为可拟合的 sklearn Pipeline。

- FeatureActionsTransformer 顺序执行删列、log1p、缺失指示和哨兵转缺失。
- ColumnTransformer 分别处理数值、类别和文本，预处理均在训练折内拟合。
- 支持逻辑回归、随机森林和直方图梯度提升。
- score_predictions 支持 macro F1、accuracy 和 ROC AUC。

输入与输出：输入 ExperimentSpec、特征表和训练种子；输出 Pipeline 或指标值。

技术要点：随机森林固定单线程，Runner 同时限制 BLAS 线程，减少资源波动。

训练调度器 budgetlab runner.py

模块定位：在主进程外执行训练并处理预算、超时、日志和持久化。

- 按用途预估拟合次数，在启动 worker 前预留实验、拟合和秒数预算。
- 把规格、契约、数据路径和用途写入 job JSON，调用 budgetlab.worker。
- 捕获 stdout stderr，处理无结果退出和 TimeoutExpired。
- 完成后按实际消耗结算，无法确认的拟合进入 unknown。

输入与输出：输入 ExperimentSpec、TaskContract、用途和切分种子；输出 ExecutionResult 与日志。

技术要点：Runner 提供进程隔离和超时控制，但不宣称是安全沙箱。

训练工作进程 budgetlab worker.py

模块定位：执行单个训练任务并确保数据哈希与契约一致。

- 重新验证 ExperimentSpec 与 TaskContract。
- 只调用 load_development，并检查 development 数据哈希。
- 探索与确认使用 StratifiedKFold；final_refit 在全部开发数据上拟合。
- 捕获异常并返回状态、每折分数、拟合次数、耗时和错误。

输入与输出：输入 Runner 写出的 job JSON；输出 result JSON 和可选 model.joblib。

技术要点：fit_attempts 与 fit_completed 分开记录，是资源核算的事实来源。

评价与判定 budgetlab evaluator.py

模块定位：计算噪声地板、同折候选比较和未见种子确认。

- calibrate 对多个基线切分分数计算样本标准差并得到 min_effect。
- evaluate_candidate 计算候选与比较器的同折差值均值。
- evaluate_confirmation 要求三个种子的差值中位数达标且正向比例至少三分之二。

输入与输出：输入真实 ExecutionResult；输出 computed_by 为 evaluator 的 EvaluationRecord。

技术要点：阈值是工程决策门槛，不应表述为统计显著性检验。

资源预算 budgetlab budget.py

模块定位：统一管理 Token、请求、成本、实验、拟合和训练秒数。

- can_reserve 进行不改变状态的预算试算。
- reserve 创建唯一 action 并把计划资源加入 reserved。
- settle 将 reserved 转为 used 和 unknown，并拒绝重复结算。
- tail_reserve 为校准、确认和最终重训保留尾部资源。

输入与输出：输入 BudgetLimits 和 ResourceVector；输出 budget.json 与是否允许执行。

技术要点：unknown 计入占用，未知消耗不会重新当作可用资源。

事件账本与原子写入 budgetlab ledger.py

模块定位：建立连续追加事件并降低中断时 JSON 损坏风险。

- atomic_write_json 使用同目录临时文件、flush、fsync 和 os.replace。
- EventLedger 分配连续 seq、UUID、UTC 时间和 run_id。
- read_events 校验事件序列连续性。

输入与输出：输入事件类型、action、artifact 和 details；输出 events.jsonl 与原子 JSON。

技术要点：当前是单进程写者，线程锁防止同进程并发冲突。

代码与环境来源 budgetlab provenance.py

模块定位：记录 run 对应的 Git 状态、Python、平台和依赖版本。

- git_info 保存提交、分支、dirty 状态和变更文件。
- environment_info 保存运行时、平台、架构和关键包版本。
- code_version_label 生成短提交号和 dirty 标签。

输入与输出：输入仓库根目录与本机环境；输出写入 run_config 和 summary。

技术要点：正式 benchmark 应来自非 dirty 提交并在冻结代码后运行。

运行报告 budgetlab reporting.py

模块定位：从程序汇总生成 Markdown 报告并保存 summary。

- 报告包含基线、探索最优、确认、交付、成本、过程统计和实验表。
- 明确说明指标由程序计算、scripted 不是 LLM、holdout 在冻结后读取。
- final_evaluation 存在时自动补入密封 holdout 分数。

输入与输出：输入 controller 生成的 summary；输出 report.md 和 summary.json。

技术要点：正式展示材料应只读取 summary 和 final_evaluation，不能手填指标。

密封最终评测 budgetlab evaluation

模块定位：对冻结模型执行一次性 holdout 评分并校验模型和数据。

- final_evaluation.json 已存在时拒绝第二次评测。
- 校验 model.joblib SHA256 与 frozen_delivery 一致。
- 校验 holdout 哈希并只选择 approved_feature_columns。
- 记录 holdout 分数、开发集差值和评测时间。

输入与输出：输入 run_id；输出 final_evaluation.json 并刷新 report.md。

技术要点：这是唯一读取 data sealed 的业务模块，应保持最小化和只读。

4 关键功能流程

4.1 数据准备

阶段	处理	主要产物
任务解析	读取 TaskDefinition 和场景卡	任务参数与字段可用性
原始加载	下载或读取原始数据并校验来源	features target
固定切分	使用 holdout_seed 分层切分并写出	development.csv holdout.csv
完整性记录	计算 SHA256、行数和正例率	manifest.json

阶段	处理	主要产物
研究加载	只加载 development 并提取 row_id	DatasetBundle

4.2 研究循环

BudgetLab 的循环是受预算和规则驱动的状态机。提案器提出两个候选，控制器先筛掉重复、非法和证据引用错误的规格，再依据新颖性、历史耗时和预算决定运行一个还是两个。未达到 min_effect 时 stagnation 增加；达到阈值后可以触发零 Token 工具搜索、Critic 和 replan。

停止条件	含义
target_reached	达到配置目标分数
max_rounds	已使用最大提案轮数
diminishing_returns	连续无有效提升达到阈值
no_valid_candidates	连续空提案或候选全部无效
budget_exhausted	无法预留下一次实验
llm_budget_exhausted	无法预留下一次提案调用
space_exhausted	随机采样找不到新规格

4.3 候选裁决与交付

9. 候选与初始基线进行同折配对比较，平均差值达到 min_effect 才是 candidate_pass。
10. 系统同时计算候选相对前一最优和其自报比较器的差值，但交付资格仍以初始基线为准。
11. 所有 candidate_pass 中按开发集分数、耗时和规格哈希选择 finalist。
12. finalist 在三个未见切分种子上重复候选与基线训练。
13. 只有 confirmed_pass 才交付 finalist；否则交付基线。
14. 交付规格在全部 development 上重训，记录模型哈希和冻结时间。

4.4 三种研究系统

系统	提案来源	调度能力	用途
random_search	SpecSampler	无轮次和 LLM，直接在同一空间采样	零 Token 对照组
budgetlab_no_schedule	scripted 或 LLM	筛选 去重 预算 停止，无工具调度	消融组
budgetlab	scripted 或 LLM	增加工具搜索 replan 和按需 Critic	完整系统

5 配置 产物 测试与运行

5.1 关键配置

配置组	主要字段	作用
任务与切分	task_id cv_folds split_seed calibration seeds confirmation seeds	决定任务与开发阶段切分
训练	training_seed search_seed min_effect_floor	控制随机性、搜索和最小工程提升
预算	max_llm_tokens requests experiments fits total_seconds	限制成本与运行规模
循环	max_rounds stagnation_tool_search replan stop	控制调度与停止
LLM	model base_url output_mode max_output_tokens retries prices	控制模型接入和资源估算

5.2 Run 目录产物

产物	内容与用途
protocol.json	校准前协议，包括任务 字段 种子 模型空间和预算
contract.json	校准后冻结契约，包括 min_effect 和切分清单哈希
scenario_card.json eda.json	业务字段边界和开发集画像

产物	内容与用途
run_config.json	原始配置 系统 提案来源 Git 与环境版本
events.jsonl budget.json	连续审计事件和预算状态
llm_calls.jsonl llm 目录	提案与 Critic 的上下文 输出 usage 和错误
rounds experiments evaluations logs	轮次决策 实验规格 训练结果 裁决和日志
confirmation.json	三个未见切分种子的确认结果
delivery	model.joblib 与 frozen_delivery.json
summary.json report.md	机器可读汇总和运行报告
final_evaluation.json	一次性密封 holdout 结果

5.3 常用命令

```
uv sync --frozen --extra dev --extra llm
python -m budgetlab.cli prepare --task bank_marketing
python -m budgetlab.cli research --config configs/bank_marketing.yaml --system random_search
python -m budgetlab.cli research --config configs/bank_marketing.yaml --system budgetlab --
proposer scripted
python -m budgetlab.cli probe-llm --config configs/bank_marketing.yaml
python -m budgetlab.cli evaluate --run-id <RUN_ID>
python -m budgetlab.cli reproduce --run-id <RUN_ID>
python -m budgetlab.cli replay --run-id <RUN_ID>
python -m pytest -q
ruff check .
```

5.4 测试覆盖

- Schema 与契约：严格字段、两个不同候选、特征动作顺序、数值域和文本兼容性。
- 预算与账本：执行前阻断、unknown 消耗、重复结算拒绝和连续事件。
- Pipeline 与 Runner：真实拟合、文本分支、确定性切分和超时核算。
- Evaluator：样本标准差噪声地板、同折比较和三种子确认。
- 控制器集成：完整 scripted 循环、一次性 holdout、复现、回放、随机零 Token、LLM 计量、重复拦截和零预算阻断。
- LLM 测试使用 Pydantic AI FunctionModel，不联网且不依赖 API Key。

6 当前限制与扩展建议

6.1 已知限制

限制	影响	建议
Bank 数据按时间排序但使用随机分层切分	未来营销效果可能偏乐观	新增时间切分并冻结为独立协议
尚无真实托管模型运行证据	不能证明 LLM 优于脚本或随机	先过 probe 门禁再跑固定预算矩阵
单机文件级 holdout 隔离	外部高权限进程仍可读取	使用独立评测服务或受限环境
训练超时依赖子进程	无系统调用级安全隔离	容器化并限制 CPU 内存 网络与文件
任务和模型空间较小	结论外推范围有限	增加任务适配器和 benchmark runner
报告以 Markdown 为主	缺少交互轨迹和成本展示	实现严格只读 UI

6.2 扩展时必须保持的约束

- 新增特征动作要同步修改 schemas、contracts、pipeline、search、proposers 的 legal_space 并补测试。
- 新增任务要增加 TaskDefinition、加载器和逐字段场景卡，再通过 prepare 生成隔离数据。
- 循环政策和预算优先修改 configs，不在控制器写任务特定默认值。
- 新资源消耗必须接入 reserve 与 settle；新结论必须关联产物并追加 ledger 事件。
- 读取 sealed holdout 的代码必须留在 evaluation 边界，并保持一次性评测。
- 正式 benchmark 前冻结代码和配置，记录 tag，并从非 dirty 提交产生结果。

6.3 推荐的下一阶段

15. 增加 benchmark.py，按系统、提案器、重复次数和预算生成运行矩阵并统一 evaluate。
16. 建立 results run_id 精简发布目录，只保留 summary、final_evaluation 和 report。
17. 实现只读展示页，分别显示研究轨迹、性能成本曲线和证据链。
18. 为真实 LLM 建立 provider 门禁：Schema 成功、usage 非空、价格可配置、失败进入 unknown。
19. 将 holdout 评测迁移到独立进程或服务，使用只读输入和一次性授权。

附录 模块依赖速查

模块	直接依赖方向	主要使用者
schemas	Pydantic	几乎全部模块
contracts	schemas	controller search proposers
tasks	schemas YAML pandas	data controller
data	tasks contracts ledger sklearn	controller worker evaluation
profile	tasks pandas	controller proposers search
pipeline	schemas sklearn pandas	worker evaluation reproduce
runner	budget ledger worker	controller
evaluator	schemas statistics	controller
proposers	schemas contracts state Pydantic AI	controller
search	schemas	controller
budget	schemas ledger	runner controller
ledger	标准库	所有写入模块
state	schemas contracts	controller proposers
reporting	summary JSON	controller evaluation
evaluation	data pipeline reporting ledger	CLI
provenance	Git metadata	controller
cli	controller data evaluation	用户与脚本

文档结论：BudgetLab 的核心不是某个单一模型，而是一套将提案、训练、裁决、预算、审计和最终评测明确分权的实验治理机制。维护系统时应优先保护契约冻结、holdout 隔离、程序裁决、资源计量和证据可追溯这五条主线。